

LAPORAN TUGAS BESAR 1
IF4021 PEMODELAN DAN SIMULASI

Simulasi Antrian Perjalanan Bus



Disusun oleh:

Kelompok 4

Ahmad Ibrahim 13523089

Carlo Angkisan 13523091

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

1	Deskripsi Masalah	1
2	Source Code	2
3	Output Program	9

1 Deskripsi Masalah

Permasalahan yang dimodelkan adalah sistem bus *shuttle* yang beroperasi di sebuah bandara dengan tiga lokasi pemberhentian, yaitu Terminal 1, Terminal 2, dan Car Rental. Sistem ini merupakan soal 2.38 pada buku *Simulation Modeling and Analysis* karya Averill M. Law (halaman 179). Bus beroperasi secara berurutan dari Car Rental, kemudian menuju Terminal 1, Terminal 2, dan kembali ke Car Rental. Jarak antar lokasi tersebut secara berturut-turut adalah 4,5 mil, 1 mil, dan 4,5 mil, sehingga dengan kecepatan bus sebesar 30 mil/jam, waktu perjalanan antar lokasi masing-masing adalah 9 menit, 2 menit, dan 9 menit.

Penumpang datang secara acak di setiap lokasi mengikuti proses Poisson dengan laju kedatangan 14 orang/jam di Terminal 1, 10 orang/jam di Terminal 2, dan 24 orang/jam di Car Rental. Penumpang dari Terminal 1 dan Terminal 2 menuju Car Rental, sedangkan penumpang dari Car Rental menuju Terminal 1 dengan probabilitas 0,583 dan Terminal 2 dengan probabilitas 0,417. Bus memiliki kapasitas 20 orang. Waktu untuk menurunkan penumpang mengikuti distribusi $U(16, 24)$ detik, sedangkan waktu untuk menaikkan penumpang mengikuti distribusi $U(15, 25)$ detik. Pada setiap lokasi, bus harus berhenti setidaknya selama 5 menit.

Sistem ini dimodelkan menggunakan *Discrete-Event Simulation* dan disimulasikan selama 80 jam. *Event* dalam simulasi meliputi kedatangan penumpang, kedatangan bus, selesai proses naik dan turun penumpang, tercapainya waktu berhenti minimum, keberangkatan bus, dan akhir simulasi. Hasil simulasi digunakan untuk menganalisis jumlah dan *delay* antrian, jumlah penumpang di dalam bus, waktu berhenti bus, waktu satu *loop*, serta waktu penumpang berada dalam sistem berdasarkan lokasi asal.

Interpretasi Masalah

Karena terdapat beberapa aspek yang tidak dijelaskan secara rinci dalam soal, simulasi ini menggunakan beberapa asumsi dan interpretasi sebagai berikut:

- Bus mulai dari Car Rental pada menit ke-0 dan langsung berangkat tanpa berhenti 5 menit terlebih dahulu. Satu *loop* dihitung sejak bus berangkat dari Car Rental hingga bus berangkat lagi dari Car Rental.
- Saat bus berhenti, penumpang yang tujuannya lokasi tersebut diturunkan terlebih dahulu, baru kemudian penumpang di antrian naik. Proses turun dan naik dilakukan satu per satu sesuai urutan kedatangan.
- Penumpang yang datang ketika bus masih berhenti di lokasinya ikut masuk antrian dan tetap boleh naik selama bus belum penuh. Bus baru berangkat apabila sudah berhenti selama 5 menit, tidak ada lagi penumpang yang turun, dan antrian sudah kosong atau bus sudah penuh.
- *Delay* di antrian dihitung sejak penumpang datang hingga ia mulai naik. Sejak mulai naik, penumpang tersebut sudah dihitung berada di dalam bus sampai selesai turun.
- Waktu di sistem dihitung sejak penumpang datang hingga ia selesai turun di lokasi tujuannya.

2 Source Code

```
1  #include "simlib.h"
2
3  #define NUM_LOCATIONS 3
4  #define CAR_RENTAL 3
5
6  #define EVENT_ARRIVAL_1 1
7  #define EVENT_ARRIVAL_2 2
8  #define EVENT_ARRIVAL_3 3
9  #define EVENT_BUS_ARRIVAL 4
10 #define EVENT_UNLOAD_DONE 5
11 #define EVENT_LOAD_DONE 6
12 #define EVENT_BUS_DEPARTURE 7
13 #define EVENT_END_SIMULATION 8
14
15 #define LIST_QUEUE(i) (i)
16 #define LIST_BUS(j) (NUM_LOCATIONS + (j))
17
18 #define ATTR_ARRIVAL_TIME 3
19 #define ATTR_ORIGIN 4
20 #define ATTR_DESTINATION 5
21
22 #define SAMPST_DELAY(i) (i)
23 #define SAMPST_STOP(i) (3 + (i))
24 #define SAMPST_LOOP 7
25 #define SAMPST_SYSTEM(i) (7 + (i))
26
27 #define TIMEST_BUS 1
28
29 #define STREAM_ARRIVAL(i) (i)
30 #define STREAM_UNLOAD 4
31 #define STREAM_LOAD 5
32 #define STREAM_DESTINATION 6
33
34 double arrival_rate[NUM_LOCATIONS + 1];
35 double prob_dest[NUM_LOCATIONS];
36 int bus_capacity;
37 double bus_speed;
38 double unload_min, unload_max;
39 double load_min, load_max;
40 double min_stop_time;
41 double sim_length;
42
43 double mean_interarrival[NUM_LOCATIONS + 1];
44
45 /* miles from location i to the next location on the route */
46 double distance_to_next[NUM_LOCATIONS + 1] = {0.0, 1.0, 4.5, 4.5};
47
48 int bus_location;
49 int bus_at_stop;
50 int bus_busy;
51 int min_stop_passed;
52 double bus_stop_start;
53 double loop_start;
54
55 FILE *infile, *outfile;
56
```

```

57 void init_model(void);
58 void arrive(int location);
59 void bus_arrive(void);
60 void unload_done(void);
61 void load_done(void);
62 void bus_depart(void);
63 void process_bus_at_stop(void);
64 int num_on_bus(void);
65 void print_stat(const char *tag, const char *label, double avg, double max, double min
    , int is_count, int has_min);
66 void write_report(void);
67 void report(void);
68
69 int main()
70 {
71     infile = fopen("data/bus.in", "r");
72     if (infile == NULL)
73     {
74         fprintf(stderr, "Failed to open data/bus.in\n");
75         exit(1);
76     }
77
78     fscanf(infile, "%lf %lf %lf", &arrival_rate[1], &arrival_rate[2], &arrival_rate
    [3]);
79     fscanf(infile, "%lf %lf", &prob_dest[1], &prob_dest[2]);
80     fscanf(infile, "%d %lf", &bus_capacity, &bus_speed);
81     fscanf(infile, "%lf %lf", &unload_min, &unload_max);
82     fscanf(infile, "%lf %lf", &load_min, &load_max);
83     fscanf(infile, "%lf", &min_stop_time);
84     fscanf(infile, "%lf", &sim_length);
85
86     for (int i = 1; i <= NUM_LOCATIONS; ++i)
87         mean_interarrival[i] = 60.0 / arrival_rate[i];
88
89     /* random_integer needs a cumulative distribution */
90     prob_distrib[1] = prob_dest[1];
91     prob_distrib[2] = prob_dest[1] + prob_dest[2];
92
93     maxatr = 5;
94     init_simlib();
95
96     init_model();
97
98     do
99     {
100         timing();
101
102         switch (next_event_type)
103         {
104             case EVENT_ARRIVAL_1:
105             case EVENT_ARRIVAL_2:
106             case EVENT_ARRIVAL_3:
107                 arrive(next_event_type);
108                 break;
109             case EVENT_BUS_ARRIVAL:
110                 bus_arrive();
111                 break;
112             case EVENT_UNLOAD_DONE:

```

```

113         unload_done();
114         break;
115     case EVENT_LOAD_DONE:
116         load_done();
117         break;
118     case EVENT_BUS_DEPARTURE:
119         bus_depart();
120         break;
121     case EVENT_END_SIMULATION:
122         report();
123         break;
124     }
125 } while (next_event_type != EVENT_END_SIMULATION);
126
127 fclose(infile);
128
129 return 0;
130 }
131
132 void init_model(void)
133 {
134     bus_location = CAR_RENTAL;
135     bus_at_stop = 0;
136     bus_busy = 0;
137     min_stop_passed = 0;
138     bus_stop_start = 0.0;
139     loop_start = 0.0;
140
141     for (int i = 1; i <= NUM_LOCATIONS; ++i)
142         event_schedule(expon(mean_interarrival[i], STREAM_ARRIVAL(i)), EVENT_ARRIVAL_1
143             + i - 1);
144
145     event_schedule(0.0, EVENT_BUS_DEPARTURE);
146     event_schedule(sim_length * 60.0, EVENT_END_SIMULATION);
147 }
148
149 void arrive(int location)
150 {
151     event_schedule(sim_time + expon(mean_interarrival[location], STREAM_ARRIVAL(
152         location)),
153         EVENT_ARRIVAL_1 + location - 1);
154
155     transfer[ATTR_ARRIVAL_TIME] = sim_time;
156     transfer[ATTR_ORIGIN] = location;
157     if (location == CAR_RENTAL)
158         transfer[ATTR_DESTINATION] = random_integer(prob_distrib, STREAM_DESTINATION);
159     else
160         transfer[ATTR_DESTINATION] = CAR_RENTAL;
161     list_file(LAST, LIST_QUEUE(location));
162
163     if (bus_at_stop && bus_location == location)
164         process_bus_at_stop();
165 }
166
167 void bus_arrive(void)
168 {
169     bus_at_stop = 1;
170     bus_stop_start = sim_time;

```

```

169     min_stop_passed = 0;
170     event_schedule(sim_time + min_stop_time, EVENT_BUS_DEPARTURE);
171
172     process_bus_at_stop();
173 }
174
175 void unload_done(void)
176 {
177     list_remove(FIRST, LIST_BUS(bus_location));
178     sampst(sim_time - transfer[ATTR_ARRIVAL_TIME], SAMPST_SYSTEM((int)transfer[
ATTR_ORIGIN]));
179     timest(num_on_bus(), TIMEST_BUS);
180
181     bus_busy = 0;
182     process_bus_at_stop();
183 }
184
185 void load_done(void)
186 {
187     bus_busy = 0;
188     process_bus_at_stop();
189 }
190
191 // void min_stop_done(void)
192 // {
193 //     min_stop_passed = 1;
194 //     process_bus_at_stop();
195 // }
196
197 void bus_depart(void)
198 {
199     if (bus_at_stop)
200     {
201         min_stop_passed = 1;
202         process_bus_at_stop();
203         return;
204     }
205
206     /* the first departure at t = 0 has no stop or loop to record */
207     if (sim_time > 0.0)
208         sampst(sim_time - bus_stop_start, SAMPST_STOP(bus_location));
209
210     if (bus_location == CAR_RENTAL)
211     {
212         if (sim_time > 0.0)
213             sampst(sim_time - loop_start, SAMPST_LOOP);
214         loop_start = sim_time;
215     }
216
217     event_schedule(sim_time + distance_to_next[bus_location] / bus_speed * 60.0,
EVENT_BUS_ARRIVAL);
218     bus_location = bus_location % NUM_LOCATIONS + 1;
219 }
220
221 void process_bus_at_stop(void)
222 {
223     if (bus_busy)
224         return;

```

```

225
226     if (list_size[LIST_BUS(bus_location)] > 0)
227     {
228         bus_busy = 1;
229         event_schedule(sim_time + uniform(unload_min, unload_max, STREAM_UNLOAD) /
230 60.0, EVENT_UNLOAD_DONE);
231     }
232     else if (list_size[LIST_QUEUE(bus_location)] > 0 && num_on_bus() < bus_capacity)
233     {
234         list_remove(FIRST, LIST_QUEUE(bus_location));
235         sampst(sim_time - transfer[ATTR_ARRIVAL_TIME], SAMPST_DELAY(bus_location));
236         list_file(LAST, LIST_BUS((int)transfer[ATTR_DESTINATION]));
237         timest(num_on_bus(), TIMEST_BUS);
238
239         bus_busy = 1;
240         event_schedule(sim_time + uniform(load_min, load_max, STREAM_LOAD) / 60.0,
241 EVENT_LOAD_DONE);
242     }
243     else if (min_stop_passed)
244     {
245         bus_at_stop = 0;
246         event_schedule(sim_time, EVENT_BUS_DEPARTURE);
247     }
248 }
249
250 int num_on_bus(void)
251 {
252     int total = 0;
253
254     for (int j = 1; j <= NUM_LOCATIONS; ++j)
255         total += list_size[LIST_BUS(j)];
256     return total;
257 }
258
259 void print_stat(const char *tag, const char *label, double avg, double max, double min
260 , int is_count, int has_min)
261 {
262     char max_str[16], min_str[16] = "-";
263
264     snprintf(max_str, sizeof max_str, is_count ? "%.0f" : "%.3f", max);
265     if (has_min)
266         snprintf(min_str, sizeof min_str, "%.3f", min);
267     fprintf(outfile, "%-4s%-38s|%.10.3f |%.10s |%.10s\n", tag, label, avg, max_str,
268 min_str);
269 }
270
271 void report(void)
272 {
273     char answer[8];
274
275     outfile = stdout;
276     write_report();
277
278     printf("\nSave output to data/bus.out? (y/n): ");
279     if (fgets(answer, sizeof answer, stdin) == NULL || (answer[0] != 'y' && answer[0]
280 != 'Y'))
281         return;

```

```

278     outfile = fopen("data/bus.out", "w");
279     if (outfile == NULL)
280     {
281         fprintf(stderr, "Failed to open data/bus.out\n");
282         return;
283     }
284     write_report();
285     fclose(outfile);
286     printf("Output saved to data/bus.out\n");
287 }
288
289 void write_report(void)
290 {
291     char label[64];
292
293     fprintf(outfile, "AIRPORT SHUTTLE BUS SIMULATION\n\n");
294
295     fprintf(outfile, "Input Parameters\n");
296     fprintf(outfile, "-----\n");
297     fprintf(outfile, "%-42s= %.1f %.1f %.1f per hour\n", "Arrival rates at locations
1, 2, 3", arrival_rate[1], arrival_rate[2], arrival_rate[3]);
298     fprintf(outfile, "%-42s= %.3f %.3f\n", "Destination probabilities from car rental
, prob_dest[1], prob_dest[2]);
299     fprintf(outfile, "%-42s= %d people\n", "Bus capacity", bus_capacity);
300     fprintf(outfile, "%-42s= %.1f miles per hour\n", "Bus speed", bus_speed);
301     fprintf(outfile, "%-42s= %.1f to %.1f seconds\n", "Unloading time per person",
unload_min, unload_max);
302     fprintf(outfile, "%-42s= %.1f to %.1f seconds\n", "Loading time per person",
load_min, load_max);
303     fprintf(outfile, "%-42s= %.1f minutes\n", "Minimum stop time", min_stop_time);
304     fprintf(outfile, "%-42s= %.1f hours\n\n", "Simulation length", sim_length);
305
306     fprintf(outfile, "Simulation Results\n");
307     fprintf(outfile, "-----\n");
308     fprintf(outfile, "Locations: 1 = Terminal 1, 2 = Terminal 2, 3 = Car rental\n");
309     fprintf(outfile, "All times are in minutes\n\n");
310     fprintf(outfile, "%-42s|%10s |%10s |%10s\n", "", "Average", "Max", "Min");
311
312     for (int i = 1; i <= NUM_LOCATIONS; ++i)
313     {
314         filest(LIST_QUEUE(i));
315         snprintf(label, sizeof label, "Number in queue at location %d", i);
316         print_stat(i == 1 ? "[a]" : "", label, transfer[1], transfer[2], 0.0, 1, 0);
317     }
318
319     for (int i = 1; i <= NUM_LOCATIONS; ++i)
320     {
321         sampst(0.0, -SAMPST_DELAY(i));
322         snprintf(label, sizeof label, "Delay in queue at location %d", i);
323         print_stat(i == 1 ? "[b]" : "", label, transfer[1], transfer[3], 0.0, 0, 0);
324     }
325
326     timest(0.0, -TIMEST_BUS);
327     print_stat("[c]", "Number on the bus", transfer[1], transfer[2], 0.0, 1, 0);
328
329     for (int i = 1; i <= NUM_LOCATIONS; ++i)
330     {
331         sampst(0.0, -SAMPST_STOP(i));

```

```

332     snprintf(label, sizeof label, "Bus stop time at location %d", i);
333     print_stat(i == 1 ? "[d]" : "", label, transfer[1], transfer[3], transfer[4],
334 0, 1);
335 }
336
337 sampst(0.0, -SAMPST_LOOP);
338 print_stat("[e]", "Bus loop time", transfer[1], transfer[3], transfer[4], 0, 1);
339
340 for (int i = 1; i <= NUM_LOCATIONS; ++i)
341 {
342     sampst(0.0, -SAMPST_SYSTEM(i));
343     snprintf(label, sizeof label, "Time in system, arrived at location %d", i);
344     print_stat(i == 1 ? "[f]" : "", label, transfer[1], transfer[3], transfer[4],
345 0, 1);
346 }

```

3 Output Program

AIRPORT SHUTTLE BUS SIMULATION

Input Parameters

Arrival rates at locations 1, 2, 3 = 14.0 10.0 24.0 per hour
 Destination probabilities from car rental = 0.583 0.417
 Bus capacity = 20 people
 Bus speed = 30.0 miles per hour
 Unloading time per person = 16.0 to 24.0 seconds
 Loading time per person = 15.0 to 25.0 seconds
 Minimum stop time = 5.0 minutes
 Simulation length = 80.0 hours

Simulation Results

Locations: 1 = Terminal 1, 2 = Terminal 2, 3 = Car rental
 All times are in minutes

	Average	Max	Min
[a] Number in queue at location 1	7.368	27	-
Number in queue at location 2	4.888	17	-
Number in queue at location 3	9.321	29	-
[b] Delay in queue at location 1	31.414	91.934	-
Delay in queue at location 2	29.820	130.860	-
Delay in queue at location 3	24.130	67.485	-
[c] Number on the bus	14.114	20	-
[d] Bus stop time at location 1	6.698	10.171	5.000
Bus stop time at location 2	5.473	9.287	5.000
Bus stop time at location 3	11.402	13.743	5.572
[e] Bus loop time	43.573	48.069	35.715
[f] Time in system, arrived at location 1	51.927	112.294	17.738
Time in system, arrived at location 2	45.544	147.004	13.929
Time in system, arrived at location 3	41.724	88.289	11.532